VOLUME II OF II — COMPLETE AUTHORIZED SELF-PENTEST

Abuser's Field Manual

Tenant 5 never pays, and doesn't know P2Less at all — only its web address. This manual assumes exactly that little, and covers everything: every tool a real abuser reaches for, from the browser's own free panel up through Burp Suite; every manipulation of dates, money, plans, tiers, connectors, buttons, sign-up, and rate limits; mass account creation with a browser automation extension; getting an OTP code without ever touching the phone it was meant for; slipping extra values into the database through the same form that's supposed to only take a name and a phone number; and a forward-looking chapter for the day face verification ships.

Companion to
Five-Tenant Readiness Dossier

Covers
Phase 5, complete

Authorized by
Hamisi Onesmus, Hamzone Technologies

Run by
You and your team

AUTHORIZATION & SCOPE

This is a self-directed test of your own production system, by its own founder and team, against test accounts created for this purpose. Every technique here targets the P2Less application layer only. If any technique appears to reach outside that — the hosting platform, the network, another real customer — stop and note it rather than continuing.

IN THIS MANUAL

COVERAGE MAP — EVERY CATEGORY YOU ASKED FOR, AND WHERE IT'S TESTED

REQUESTED CATEGORY	COVERED IN
Dates	Real request tampering, Technique 1
Money / payment amounts	Vector 4; Real request tampering, Technique 3
Plans / tiers	Vector 1, 2, 5; Real request tampering, Technique 2
Connectors / channels	Vector 3, 5; Re-enabling a disabled button or channel
Buttons disabled by the UI	Re-enabling a disabled button or channel
Login and sign-up abuse	Vector 2, 10; Suspended — trying to get back in
Rate limits	Vector 8
OTP / verification bypass	Vector 9
Inserting values into the database via a form	Vector 9, second half
Automation extensions for mass account creation	Vector 10

REQUESTED CATEGORY	COVERED IN
Face verification (future)	Vector 11
Geography / country restriction	Vector 7
Session, identity, and suspension	Real request tampering, Technique 4; Suspended chapter
Self-inflating a message/AI balance	Vector 12

BEFORE YOU START

What you need, and nothing else

A computer, one web address, and a few hours spread across sessions. Every tool named anywhere in this manual is free, and every install step is spelled out when it's first needed.

WHAT YOU NEED

- A Windows, Mac, or Linux computer with an internet connection, and the Chrome browser.
- The one address you'll need for almost everything: `https://p2less-app-production.up.railway.app`
- Nothing pre-installed. Every tool this manual uses is introduced, explained, and installed step by step exactly where it's first needed — you never need to go hunting for something the manual hasn't told you to get yet.

WHAT "ABUSE TESTING" ACTUALLY MEANS HERE

You are going to behave exactly like a real person trying to get more out of P2Less than they paid for, see data they shouldn't, keep using a switched-off feature, or create accounts faster than a human normally could. Nothing here requires programming knowledge — every technique reduces to: open a panel, look at what's there, change one thing, watch what happens, write it down.

THE TOOLBOX

Every tool, explained before you touch one

Real abusers don't use one tool — they reach for whichever one fits the moment. This is the complete list used anywhere in this manual, grouped by what each one is actually for, so you know why you're opening it before you open it. Deep, hands-on install-and-use instructions for each follow in the next three chapters; this page is the map.

BUILT INTO THE BROWSER — NOTHING TO INSTALL

Chrome DevTools INSPECT + INTERCEPT

A panel built into Chrome itself (`F12`). Shows every request the page sends and lets you read and edit the page's own on-screen content.

Used for: reading responses, editing cookies, editing and resending one request at a time, removing a disabled button's restriction.

Incognito window CLEAN SLATE

`Ctrl+Shift+N`. A browser window with no saved cookies or history from your normal browsing.

Used for: testing sign-up as a "fresh" visitor without your real session interfering.

TRAFFIC INTERCEPTION & EDITING

Burp Suite Community Edition PROXY TOOL

A separate program that sits between your browser and the internet, showing and letting you edit every request before it leaves your machine.

Used for: everything DevTools does, but built for doing it repeatedly and fast (Repeater), plus automated brute-forcing (Intruder).

`portswigger.net/burp/communitydownload`

OWASP ZAP PROXY TOOL

A free, fully open-source alternative to Burp, built by the OWASP security foundation. Works the same way — a local proxy that intercepts and lets you replay requests.

Used for: a second opinion tool, useful if you want to cross-check a Burp finding with an entirely different program.

`zaproxy.org`

CALLING THE SERVER DIRECTLY, NO BROWSER AT ALL

Postman API CLIENT

A desktop app for building and firing off web requests by hand, with a friendly form-style interface instead of raw text.

Used for: replaying a captured request outside the browser entirely, useful once you already know exactly what a request looks like and just want to vary it quickly.

`postman.com/downloads`

cURL COMMAND LINE

A tiny command-line program, already installed on most computers, that sends one web request per command you type.

Used for: the fastest way to replay something you copied as "Copy as cURL" from DevTools or Burp.

already on your machine — try typing `curl --version`

LIGHTWEIGHT BROWSER EXTENSIONS

Cookie-Editor EXTENSION

A small Chrome extension giving a simple list-and-edit view of a site's cookies, quicker than DevTools' Application tab for a fast check.

Used for: viewing and editing your session cookie in one click, no panel-hunting.

`Chrome Web Store, search "Cookie-Editor"`

ModHeader EXTENSION

Adds, removes, or overrides request headers on every request your browser sends, automatically, without editing each one by hand.

Used for: spoofing things like an apparent origin IP header, testing whether P2Less trusts a client-supplied location hint.

`Chrome Web Store, search "ModHeader"`

User-Agent Switcher EXTENSION

Changes what your browser claims to be (Chrome on Windows, Safari on iPhone, etc.) without actually using a different browser or device.

Used for: checking whether any restriction is (incorrectly) keyed off device/browser claims rather than real server-side checks.

`Chrome Web Store, search "User-Agent Switcher"`

AUTOMATION — DOING ONE THING MANY TIMES, FAST

Selenium IDE EXTENSION

A free, official browser extension that records your clicks and typing as a re-playable "macro," and can loop that macro over a list of different values — no coding required.

Used for: the exact tool a real abuser installs to mass-create accounts — record one sign-up, then replay it dozens of times with a new email/phone each time.

`Chrome Web Store, search "Selenium IDE" (official, by SeleniumHQ)`

IDENTITY & LOCATION MASKING

VPN app (e.g. ProtonVPN) NETWORK

Routes your internet connection through a server in another country, so sites see that server's location and IP address instead of your own.

Used for: testing a Kenya-only restriction from outside Kenya.

`protonvpn.com/download`

Temp-mail service WEBSITE

A website that hands out a disposable, real, working email address good for a short time, with no sign-up of its own.

Used for: mass account creation testing — a fresh, real inbox for every fake tenant without using your own email over and over.

`e.g. temp-mail.org — any similar site works`

SMS-receive-online service WEBSITE

A website offering shared, public phone numbers that anyone can view incoming SMS messages sent to — including a real OTP code, if a site sends one.

Used for: testing whether P2Less's phone verification can be satisfied by a number the "owner" doesn't actually control (see Vector 9).

e.g. receive-sms-online.info — any similar site works; note these numbers are public, anyone can read messages sent to them

DIAGNOSTIC / LEAK-CHECK SITES

ipinfo.io WEBSITE

Shows the IP address and country your connection currently looks like from the outside.

Used for: confirming a VPN actually changed your apparent location before you rely on it.

browserleaks.com/webrtc WEBSITE

Checks whether your browser is leaking your real IP address through a side channel even while a VPN is active.

Used for: figuring out why a VPN didn't fully mask you, if it didn't.

TOOLS THIS MANUAL DELIBERATELY DOES NOT WALK THROUGH

Real-world account farms sometimes use paid, third-party CAPTCHA-solving services to get past human-verification puzzles at scale. P2Less doesn't currently use CAPTCHA anywhere in the flows this manual tests, so there's nothing to test against yet, and this manual won't provide a how-to for a service whose only real use is enabling fraud elsewhere. If P2Less ever adds CAPTCHA to sign-up, that's worth a dedicated follow-up test on its own, scoped the same careful way as everything else here.

TOOL DEEP DIVE, PART 1

Chrome DevTools, from zero

DevTools is a panel built into Chrome itself — nothing to download. It lets you see every request the page sends, and edit the page's own content on screen. This chapter explains every part of it you'll use later, before you need it.

Opening it

1 **YOU — DEVTOOLS**

With Chrome open on any page, press `F12` on your keyboard. (If nothing happens, right-click anywhere on the page and choose **Inspect** from the menu.)

> **Expect:** a panel opens, docked to the side or bottom, with tabs across the top: **Elements**, **Console**, **Sources**, **Network**, **Application**, and more.

2 **YOU — DEVTOOLS**

If it feels cramped, click the three dots (⋮) in DevTools' own top-right corner and pick a dock position that gives you more room, or pop it into its own window.

Pressing F12 — the real page stays on the left, DevTools opens as a dark panel alongside it.

The Elements tab — editing what's on screen

1 **YOU — DEVTOOLS**

Click **Elements**. You'll see the page's raw structure — a tree of tags like `<button>`, `<div>`, `<input>`, matching what's on screen.

2 **YOU — DEVTOOLS**

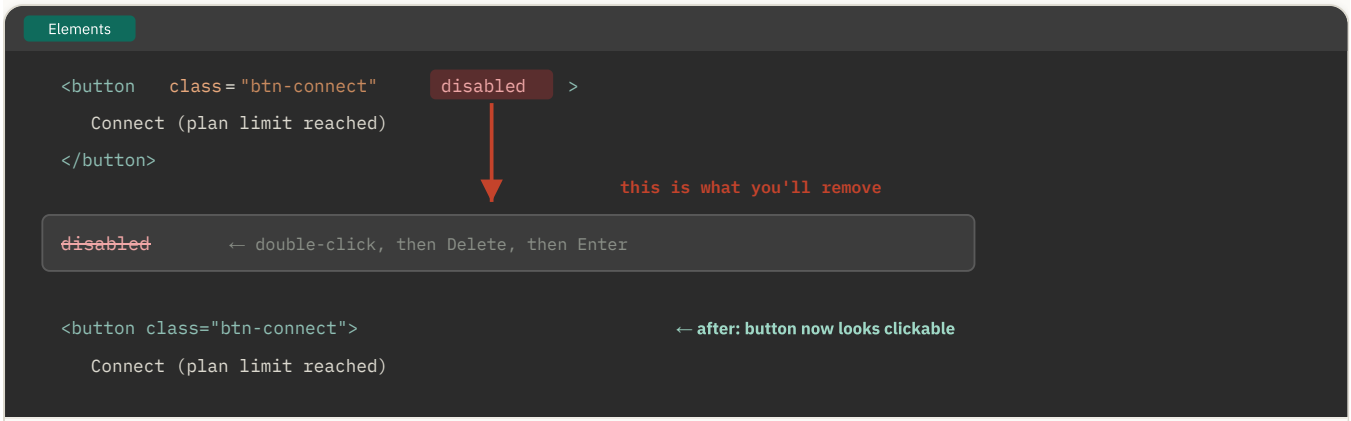
Click the element picker icon (top-left of the panel, a cursor pointing at a square; shortcut `Ctrl+Shift+C`), then click anything on the actual page.

> **Expect:** the Elements tree jumps to and highlights the exact tag for what you clicked.

3 **YOU — DEVTOOLS**

With a tag selected, double-click any attribute (a `word="value"` pair, or a bare word like `disabled`) to edit it, or right-click the tag → **Edit as HTML** to rewrite it freely. Press `Enter` or click away to apply.

> **Expect:** the change is temporary and visible only to you — reloading undoes it. It never touches the real server.



The Elements tab, before and after removing a `disabled` attribute — this is what the "re-enabling a button" chapter actually looks like.

The Network tab — watching what the page sends

1 **YOU — DEVTOOLS**

Click **Network**. Tick **Preserve log** near the top — without it, the list clears on every navigation.

2 **YOU — DEVTOOLS**

Click around P2Less normally.

> **Expect:** every action adds rows — each one a request your browser sent and the reply it got.

3 **YOU — DEVTOOLS**

Columns: **Name**, **Status** (the reply code — see the cheat sheet below), **Type**, **Size**, **Time**.

4 **YOU — DEVTOOLS**

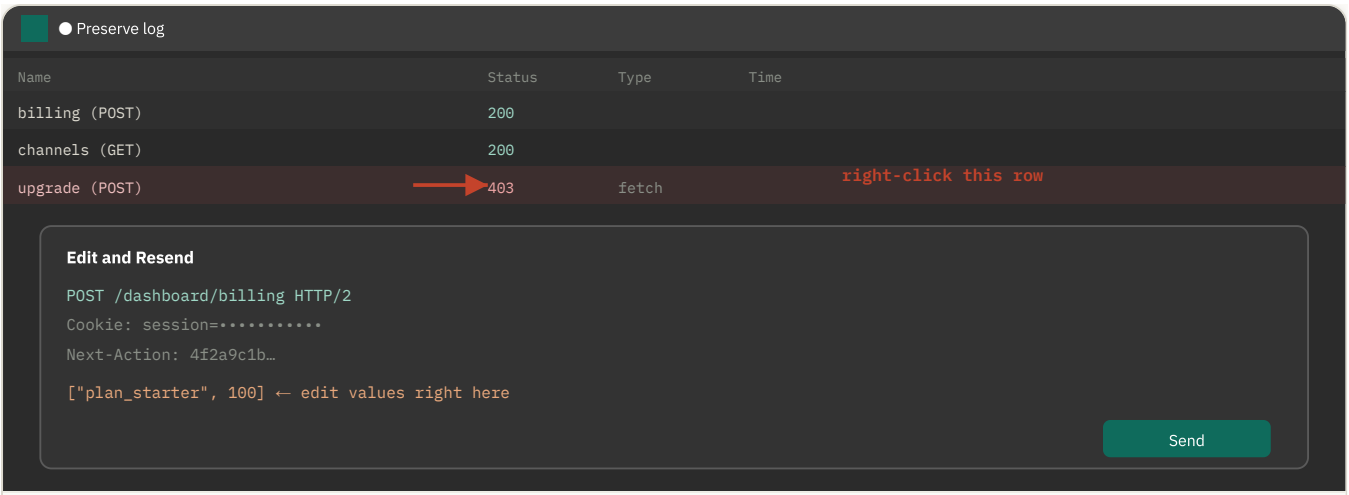
Click a row.

> **Expect:** a detail panel with **Headers** (method, URL, your session cookie), **Payload**/**Request** (what you sent), **Response** (what came back, often readable text).

5 **YOU — DEVTOOLS**

Right-click a row → **Edit and Resend**.

> **Expect:** the whole request opens as editable plain text. Change anything, click **Send** at the bottom — the result appears as a new row in the list, exactly as if triggered from the real page.



The Network tab's row list, and the Edit-and-Resend panel it opens — this exact panel is used constantly from here on.

The Application tab — cookies and stored session data

1 **YOU — DEVTOOLS**

Click **Application** (labeled **Storage** if the panel is narrow — click the `»` at the end of the tab row if you don't see it).

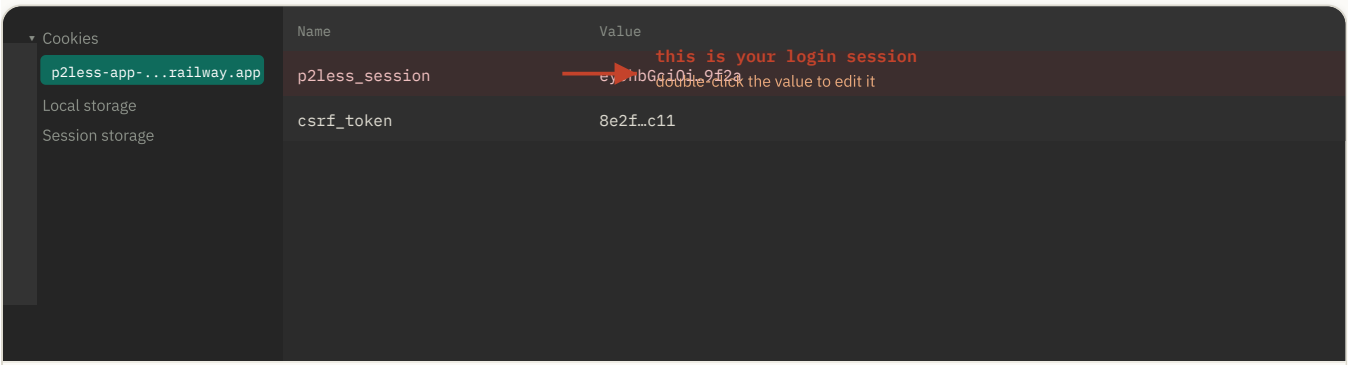
2 **YOU — DEVTOOLS**

Expand **Cookies** in the left list, click the one entry underneath it.

> **Expect:** a table of every cookie stored for the site, including your login session (named something with "session" or similar).

3 **YOU — DEVTOOLS**

Double-click a cookie's Value to edit it, or right-click a row → **Delete** to remove it. Both apply the moment you click away or reload.



Application → Cookies — the session cookie row is the one this manual asks you to edit or delete in several later chapters.

How to tell "it worked" from "it was blocked" — keep coming back to this

READING THE STATUS COLUMN (NETWORK TAB, BURP, ZAP, POSTMAN — SAME EVERYWHERE)

- `200` / `201` / `204` — succeeded. If this is a step that should have been refused, that's a real finding.
- `302` / `307` — "go look somewhere else." Open **Headers** and check the `Location` value; a redirect to `/login` after an action usually means you were treated as not authenticated.
- `400` — rejected as malformed. Usually your edit broke the format — keep the structure intact and try again.
- `401` — not logged in / session rejected. Exactly what you want after tampering with a cookie.
- `403` — logged in, but not allowed. Exactly what you want after trying another tenant's data or a restricted action.
- `404` — nothing at that address/id at all. A good, safe outcome when probing another tenant's id.
- `429` — "too much, slow down." A rate limit — central to Vector 8.
- `500` / `502` / `503` — the server broke handling your request. Always worth reporting, even though it's a different kind of bug.

READING THE RESPONSE TAB ITSELF

- A genuine block usually says something specific and human ("Your account is suspended," "This plan does not include this channel"). A vague or success-shaped response is the sign something didn't actually get checked.
- If the Response looks like a whole web page (`<!DOCTYPE html>`) rather than a short message, you were likely redirected rather than genuinely refused — check the Status code to confirm.
- Always cross-check against what actually changed on screen. A `200` and a page still showing the old, correct state is good. A `200` and a page now showing something it shouldn't is the bug.

TOOL DEEP DIVE, PART 2

Burp Suite, from zero

Burp does the same core thing as DevTools' Network tab — shows every request, lets you edit and resend — but is built for doing that fast and repeatedly, and adds a dedicated brute-force tool DevTools doesn't have.

STEP A — INSTALL IT

1 **YOU — BURP SUITE**
Go to `portswigger.net/burp/communitydownload` — the official publisher. Only ever download Burp from here.

2 **YOU — BURP SUITE**
Choose your operating system, download, and run the installer with default options — Next, Next, Install, Finish.

STEP B — LAUNCH IT AND START A PROJECT

1 **YOU — BURP SUITE**
Open Burp Suite. Accept the terms, then choose **Temporary project** → **Next** → **Use Burp defaults** → **Start Burp**.

> **Expect:** the main window opens with tabs: Dashboard, Target, Proxy, Intruder, Repeater, and more.

STEP C — OPEN BURP'S BUILT-IN BROWSER (THE EASY PATH)

WHY THIS PATH
Burp can be pointed at ordinary Chrome via manual proxy settings and a certificate install, but that's extra setup with more to get wrong. Burp ships its own already-wired-in browser window — use that.

1 **YOU — BURP SUITE**
Click **Proxy** → **Intercept**, then click **Open Browser**.

> **Expect:** a separate browser window opens. Everything inside it is automatically captured by Burp.

2 **YOU — BURP SUITE**
In that window, go to P2Less and sign in as Tenant 5.

STEP D — INTERCEPT AND EDIT A LIVE REQUEST

1 **YOU — BURP SUITE**
On **Proxy** → **Intercept**, make sure it reads **Intercept is on**.

2 **YOU — BURP SUITE**

Click any submit button on P2Less in the Burp browser.

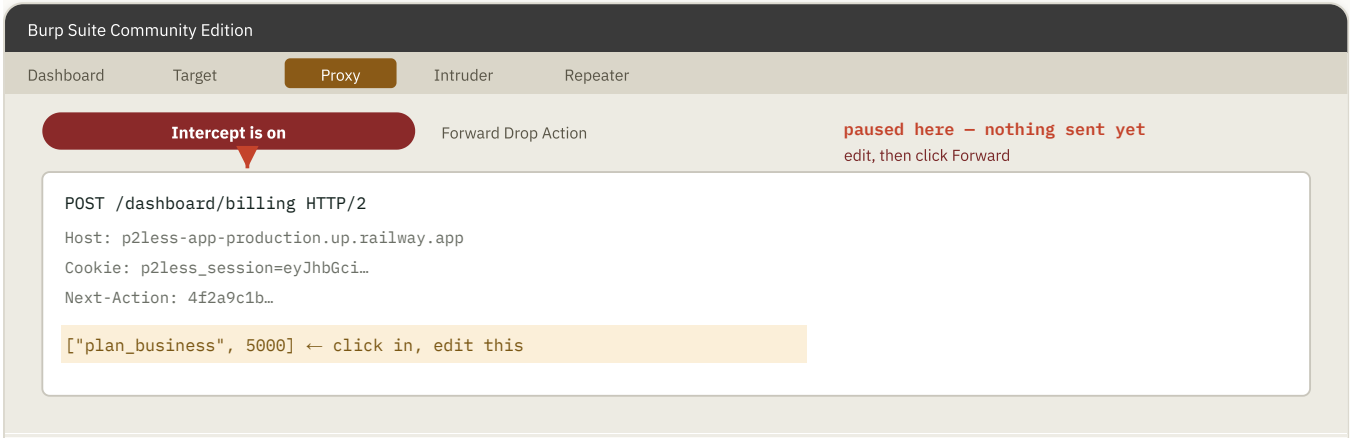
> **Expect:** the page freezes. Switch to the main Burp window — the request sits paused, raw and editable: a method and URL on the first line, headers below, and a body under a blank line if present.

3 **YOU — BURP SUITE**

Edit anything, then click **Forward** to send it on, or **Drop** to cancel it.

4 **YOU — BURP SUITE**

Turn **Intercept** back **off** when you're done editing one at a time, or every click will pause and wait.



Burp's Intercept tab — a real request paused mid-flight, editable before it ever reaches P2Less's server.

STEP E — REPEATER, FOR EDITING THE SAME REQUEST MANY TIMES

1 **YOU — BURP SUITE**

Click **Proxy → HTTP history** for a full list of everything Burp has seen.

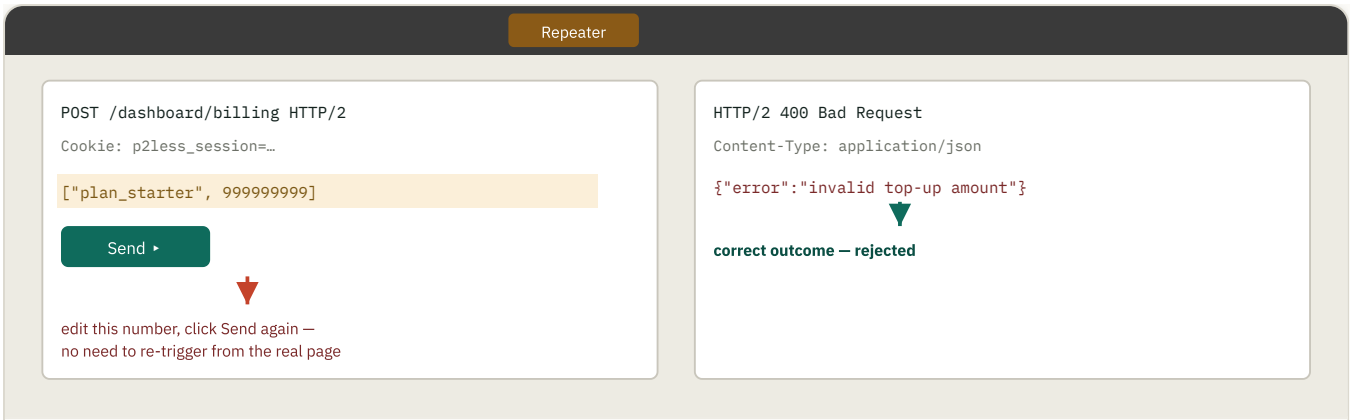
2 **YOU — BURP SUITE**

Right-click the request you want → **Send to Repeater**, then click the **Repeater** tab.

> **Expect:** the request, editable on the left; the response appears on the right after you click **Send**.

3 **YOU — BURP SUITE**

Edit, send, read the response, edit again — as many times as you like, without re-triggering it from the real page each time.



Repeater — edit on the left, response on the right, over and over without leaving this screen. Here, an inflated top-up amount is correctly rejected.

STEP F — INTRUDER, FOR BRUTE-FORCING AND RATE-LIMIT TESTING

WHAT INTRUDER IS FOR

Repeater sends one edited request at a time, by hand. Intruder automates that: give it a request with one value marked as a "payload position," give it a list of values to try there, and it fires the request once per value, collecting every response. It's the right tool whenever "try many things quickly" is the actual test — brute-forcing an OTP code, or hammering a login form to see when it starts refusing you. Community Edition's Intruder is deliberately throttled (slower than the paid version) — that's fine for this manual; it still proves the point.

1 YOU — BURP SUITE

In **Proxy** → **HTTP history**, right-click the request you want to automate → **Send to Intruder**, then click the **Intruder** tab.

2 YOU — BURP SUITE

On the **Positions** sub-tab, you'll see the request with `§` marks Burp guessed around likely values. Click **Clear §** to remove all of them, then highlight just the one value you actually want to vary (e.g. a 6-digit code) and click **Add §** so only that one is marked.

3 YOU — BURP SUITE

On the **Payloads** sub-tab, choose payload type **Numbers**, set **From** `0`, **To** `999999`, **Step** `1`, and a **Min integer digits** of `6` so every value is padded to six digits.

4 YOU — BURP SUITE

Click **Start attack** (top-right).

> **Expect:** a results window listing every attempt with its own Status column and response length. Sort by **Status** or **Length** — a successful guess almost always stands out as the one row with a different status code or a noticeably different response length than all the others.

Intruder — attack results

#	Payload	Status	Length
1	000000	400	1,204
2	000001	400	1,204
…			
14	000013	429	340
15–1,000,000		still 429…	

rate limit kicked in here
— good: it stopped a brute force early

Sort by clicking the Status or Length column header — the odd one out is usually the finding.

Intruder's results table, one row per attempt — this is what a working rate limit looks like: a wall of `429`s after a handful of tries.

TOOL DEEP DIVE, PART 3

The rest of the toolbox, from zero

Quick install-and-use notes for everything else on the map. None of these are required for every vector — each later chapter tells you exactly which tool to reach for.

OWASP ZAP

1 **YOU — ZAP**
Go to `zaproxy.org`, download the installer for your OS, run it with default options.

2 **YOU — ZAP**
Open ZAP, choose **I do not want to persist this session** for a quick one-off test.

3 **YOU — ZAP**
Click the compass-icon button in the toolbar labeled **Manual Explore**, type P2Less's address, click **Launch Browser** — ZAP opens its own Firefox window, already wired in, exactly like Burp's built-in browser.

4 **YOU — ZAP**
Everything you click in that window appears in ZAP's **History** tab at the bottom; right-click any entry → **Open/Resend with Request Editor** to edit and refire it, the ZAP equivalent of Burp's Repeater.

POSTMAN

1 **YOU — POSTMAN**
Go to `postman.com/downloads`, install, open the app.

2 **YOU — POSTMAN**
Back in DevTools' Network tab, right-click a captured request → **Copy** → **Copy as cURL**.

3 **YOU — POSTMAN**
In Postman, click **Import** (top-left), paste the copied text, click **Import** again.

> **Expect:** the request appears fully rebuilt in Postman's editor — method, URL, headers, body all filled in and independently editable, with a **Send** button and a response panel below it.

CURL

1 **YOU — CURL**
Open a terminal (on Windows, PowerShell). Copy a request from DevTools as described above.

2 **YOU — CURL**
Paste the whole copied command into the terminal and press `Enter`.

> **Expect:** the raw response text prints directly in the terminal — useful for quick, scriptable repeats without any app window at all.

COOKIE-EDITOR, MODHEADER, USER-AGENT SWITCHER

1 **YOU — EXTENSIONS**

Go to the Chrome Web Store (`chrome.google.com/webstore`), search each by name, click **Add to Chrome** for each you want.

2 **YOU — EXTENSIONS**

Each adds a small icon next to Chrome's address bar. Click it on any P2Less page to open its own simple panel — Cookie-Editor shows a list of cookies with edit/delete buttons per row; ModHeader shows a list of header rules you can toggle on and off; User-Agent Switcher shows a dropdown of device/browser identities to pick from.

SELENIUM IDE

Full walkthrough with a real recorded macro is in Vector 10 — installed there, where it's actually used.

VPN, TEMP-MAIL, SMS-RECEIVE-ONLINE, AND THE LEAK-CHECK SITES

Full walkthroughs are in Vector 7 (VPN) and Vector 9/10 (temp-mail, SMS-receive-online) — installed and used right where they're needed rather than in advance.

SETUP

Setting up Tenant 5

An entirely ordinary signup — the abuse starts after.

1 YOU

Open Chrome and go to `p2less-app-production.up.railway.app/onboard`.

> **Expect:** a page titled "Your conversational platform, ready in minutes."

2 YOU

Fill **Organization name** with `Tenant 5 — Abuse Test`, **Your phone number** with something like `+254700111005`, and your name/email, then click **Create my workspace**.

3 YOU

The next screen shows an orange "Demo only" box with a real 6-digit code already printed on it. Type that exact code into the **6-digit code** field and click **Verify and create workspace**.

> **Expect:** you land signed in, on the dashboard, with a 7-day trial — identical to every honest tenant so far.

4 YOU — DEVTOOLS

Press `F12`, click **Network**, tick **Preserve log**. Leave this open for the whole rest of this manual.

VECTOR 1

Never choose a plan

1 **YOU**

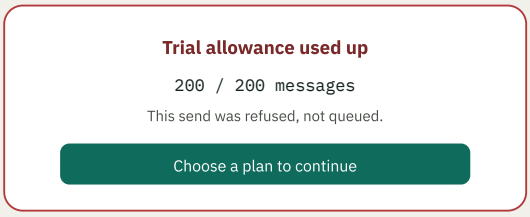
Do nothing — leave Tenant 5 on the trial exactly as created.

2 **YOU**

Connect a channel and send messages right up to the trial's 200-message / 100-AI-request ceiling, then keep going past it.

> **Expect:** once either ceiling is hit, further sends are refused with a clear, specific reason — never a silent failure and never an uncapped continuation.

what "refused, not silently uncapped" looks like



Hitting the trial ceiling — a real, specific block, not a silent pass-through.

VECTOR 2

Create a second account to dodge the trial

1 **YOU**

Once Tenant 5's trial is exhausted or expired, open an Incognito window (`Ctrl+Shift+N`) and go to `/onboard` again.

2 **YOU**

Sign up as `Tenant 5b` , reusing the exact same phone number and/or email as Tenant 5.

> **Expect:** the system recognizes the reused identity and blocks, merges, or flags the signup — never silently hands out a second, fully fresh trial.

.../onboard (Incognito window)

What catching a reused identity should look like on the sign-up form itself.

3 **YOU**

If that's blocked, try once more with a genuinely different phone and email but the same organization name and browser.

> **Note:** a real abuser has unlimited phone numbers. If nothing catches this harder case, log it as a real, open gap rather than a test failure. Vector 10 pushes this same idea to real scale.

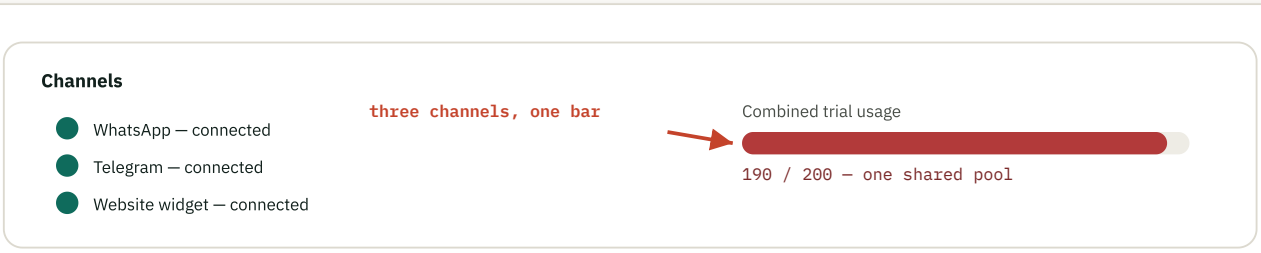
VECTOR 3

Overload the free tier

1 **YOU**

Connect every channel type to this one trial tenant — WhatsApp, Telegram, widget — and send bursts on all of them at once.

> **Expect:** combined usage across every channel counts against one single trial ceiling, not a separate free allowance per channel.



Correct pooling — three channels firing at once, but one shared trial ceiling underneath.

VECTOR 4

Fake or delay payment

1 **YOU**

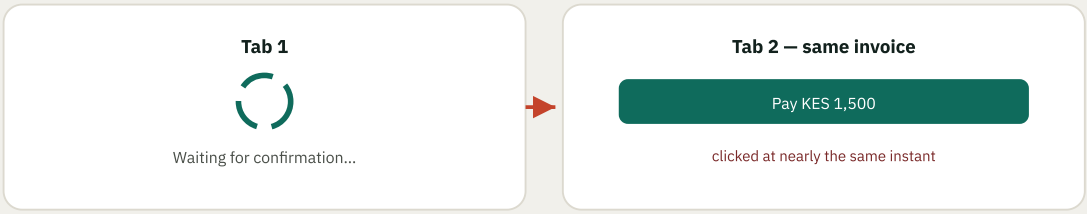
Open the upgrade modal, start an M-Pesa STK payment, then close the tab before it resolves either way.

> **Expect:** reopening billing later shows the true current state — never stuck "processing," never a plan change that didn't actually happen.

2 **YOU**

Start a payment, and while it's "waiting," open a second tab and try to trigger the same invoice's payment a second time.

> **Expect:** no double-charge, no way to end up with two successful payments against one invoice.



The double-submit race — two tabs, one invoice, fired as close together as you can manage.

VECTOR 5

Push functional limits

1 **YOU**

Try adding far more team members, channels, or connectors than Tenant 5's plan should allow.

> **Expect:** a clear "your plan allows up to N" message, never a silent allowance or a server error.

A correctly specific limit message — names the actual number, not a generic failure.

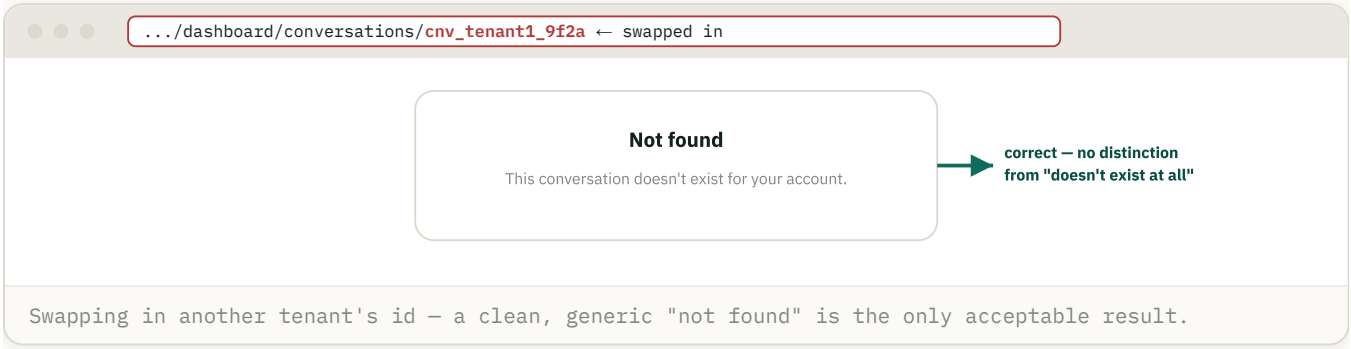
VECTOR 6

Probe for other tenants' data

1 **YOU**

While logged in as Tenant 5, edit the browser's address bar to a dashboard URL containing another tenant's id (copy one from Tenant 1's session while testing it, paste it in here).

> **Expect:** a clean "not found" or "forbidden" — never another tenant's real data rendering on screen. This is the single most serious class of bug this whole manual exists to catch.



Swapping in another tenant's id — a clean, generic "not found" is the only acceptable result.

VECTOR 7

Access from outside Kenya, undetected

If P2Less ever restricts access to Kenya only, this chapter tests whether that restriction holds against an ordinary person with a VPN — and, if it catches them, whether they can escape it.

STEP A — ESTABLISH YOUR REAL, UNMASKED BASELINE FIRST

1 **YOU**

Go to `ipinfo.io`. Write down the **Country** it reports right now — your honest starting point.

2 **YOU**

Without changing anything, try signing up as `Tenant 7a — Outside Kenya, unmasked`.

> **Expect (if a restriction exists):** a specific, honest message. **Expect (if none exists yet):** signup succeeds — note this plainly; it means the restriction isn't enforced yet, which is itself the finding.

STEP B — INSTALL A VPN

1 **YOU — VPN**

Go to `protonvpn.com/download`, download, install with default options.

2 **YOU — VPN**

Open the app, **Create Account** with a throwaway email, choose the **Free** plan, log in.

STEP C — CONNECT AND VERIFY THE MASK ACTUALLY WORKED

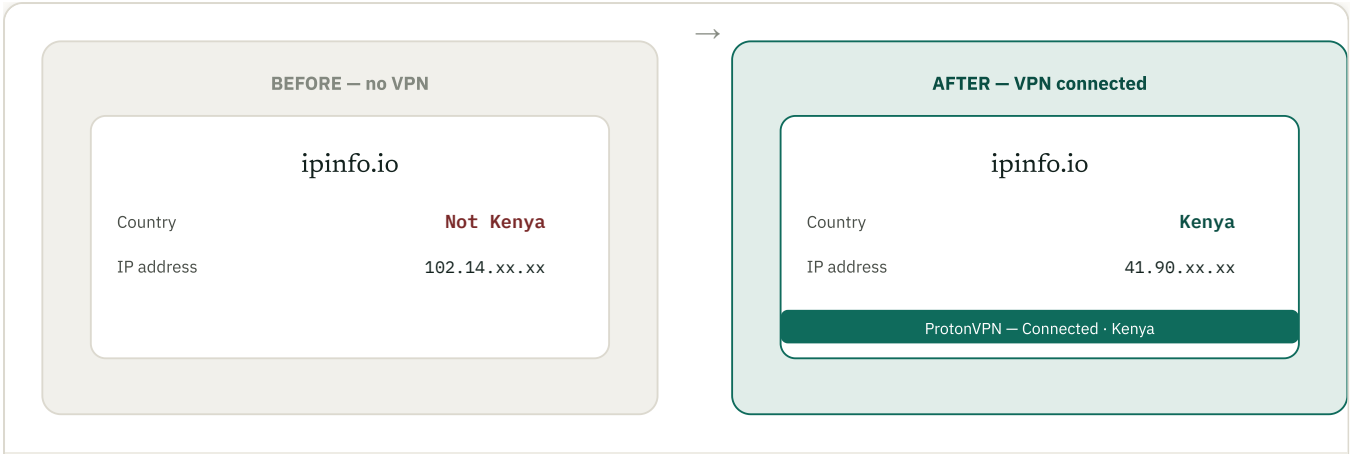
1 **YOU — VPN**

Pick Kenya if listed; otherwise the closest available African country, noted honestly.

2 **YOU**

Reload `ipinfo.io`.

> **Expect:** Country now shows the VPN's country and a new IP. If it still shows your real country, the VPN isn't actually connected.



What "the mask actually worked" looks like — the same ipinfo.io check, before and after connecting.

STEP D — MASK THE SIGNALS A VPN DOESN'T COVER

1. **YOU — WINDOWS**
   Start → type `Date and time settings` → turn off **Set time zone automatically** → set it to **(UTC+03:00) Nairobi**.

2. **YOU — CHROME**
   `chrome://settings/languages` → **Add languages** → add **English (Kenya)**, move it to the top.

3. **YOU**
   Optional: use a real Kenyan (+254) phone number if you legitimately have access to one, for the extra realism.

STEP E — TRY AGAIN, MASKED

1. **YOU**
   Fresh Incognito window, VPN still connected, sign up as `Tenant 7b — Outside Kenya, masked`.

   > **Expect & how to read it:** use the status-code cheat sheet. A clean sign-up with everything masked means the restriction can be defeated with a free VPN and a clock change. A block that still appears is a stronger, layered restriction worth documenting as such.

STEP F — IF IT CATCHES YOU ANYWAY: TRY TO ESCAPE IT

1. **YOU**
   Go to `browserleaks.com/webrtc` while connected to the VPN.

   > **Expect to find:** some browsers leak your real IP through WebRTC even with a VPN active. If this page shows your real IP alongside your VPN one, that leak may be exactly what exposed you.

2. **YOU — CHROME**
   If a leak shows up, install a "WebRTC leak block" extension, enable it, reload the leak-check page to confirm it's fixed.

3. **YOU — VPN**
   Disconnect and reconnect to a different server. Recheck `ipinfo.io` for a new IP.

4 YOU — DEVTOOLS

Application → Storage → Clear site data for P2Less's address.

5 YOU

Retry as `Tenant 7c`, fresh IP, cleared browser.

> **Expect:** if this works after Steps 1–4 but the first masked attempt didn't, the restriction is IP/fingerprint-based and rotates around fairly easily. If still blocked, it's tied to something sturdier — also a good outcome, and worth documenting as such.

6 YOU

If an already-signed-up account gets flagged mid-use instead, run the whole **Suspended — trying to get back in** chapter against it rather than duplicating those steps here.

VECTOR 8

Rate limits and brute force

Every place P2Less accepts an attempt — a login, an OTP code, a message send — should eventually say "slow down" if hit fast enough. This is where you find out if it actually does.

1 **YOU — BURP SUITE**
Set up Burp exactly as in the Burp deep-dive chapter, with its browser open and Tenant 5 logged out, sitting on the login page.

2 **YOU — BURP SUITE**
Submit one deliberately wrong login attempt (right email, wrong password) so Burp captures the request. Find it in **Proxy** → **HTTP history**, right-click → **Send to Intruder**.

3 **YOU — BURP SUITE**
On **Positions**, clear the automatic marks and mark only the password field. On **Payloads**, load a short list of 20–30 plausible guesses (or Burp's built-in **Simple list** payload type with a handful typed in by hand). Click **Start attack**.

> **Expect:** after some number of attempts — ideally in the single digits or low tens — responses should start coming back with a `429` status, or the response text should start saying something like "too many attempts," rather than silently continuing to check each guess forever.

4 **YOU — BURP SUITE**
Repeat the same idea against the **OTP verification** request from sign-up (see Vector 9 for the exact request to target) and against sending a message through a connected channel — rapid-fire the same send request many times in a row via Repeater's **Send** button clicked repeatedly, or Intruder with a list of near-identical message bodies.

> **Expect:** the same pattern — a real ceiling that kicks in, not indefinite silent acceptance.

The human-facing side of a working rate limit, on the login form itself — not just Intruder's raw table.

WHAT "GOOD" LOOKS LIKE HERE

- A rate limit exists and triggers within a reasonably small number of attempts, not thousands.
- The limit is enforced by the server (still triggers no matter which tool sends the requests), not just by the UI disabling a button after one click.
- The limit's response is honest (a real `429` or a clear message), not a silent drop that just looks the same as success.

VECTOR 9

OTP without the phone, and slipping extra values into the database

Two related tests: can verification be satisfied without ever actually receiving the code, and can the same form be used to write values into the database that a signup form was never meant to control?

Part 1 — getting past OTP without receiving it

1 **YOU**

Go to `receive-sms-online.info` (or any similar site) and pick any listed public number. These numbers are genuinely public — anyone can view messages sent to them, which is exactly the property being tested here.

2 **YOU**

Start a P2Less signup using that public number as **Your phone number**.

> **Expect:** P2Less's real SMS provider is not yet configured in production, so today the OTP is shown directly on screen in the "Demo only" box rather than actually sent anywhere — meaning this specific technique can't be meaningfully tested until real SMS delivery is live. Note this plainly as "not yet testable" rather than skipping it silently; re-run this exact step the day real SMS sending ships, checking whether the code sent to a public number can be read by anyone who visits that number's public inbox page.

3 **YOU — DEVTOOLS**

Regardless of the above, check for a leak in the meantime: with the Network tab open and Preserve log on, go through signup up to the OTP screen, then look through every captured request's **Response** tab so far — not just what's shown on screen.

> **Expect:** the real code should never appear inside a network response that reaches the browser before or independently of the clearly-labeled demo box. If you find it in a response the demo box didn't already show you, that's a real leak — the code is reaching the client through a channel that isn't honestly labeled as insecure.

4 **YOU — BURP SUITE**

Use Intruder against the OTP verification request itself (see the Intruder walkthrough in the Burp deep-dive chapter) — Numbers payload, `000000` to `999999`, 6 digits — and start the attack.

> **Expect:** Vector 8's rate limit should kick in long before this brute force could realistically succeed. If it doesn't, a 6-digit code is only 1,000,000 possibilities — genuinely guessable by brute force in a realistic amount of time without one.

5 **YOU**

Try skipping the OTP screen entirely: after starting a signup but before entering any code, edit the browser's address bar directly to whatever URL the dashboard normally lives at.

> **Expect:** refused / redirected back to verification — the account must not be usable until the code is actually confirmed server-side.

What a public SMS-receiving inbox looks like — the thing that makes phone-number verification meaningless once real SMS is live.

Part 2 — slipping extra values into the database (mass assignment)

THE IDEA IN PLAIN TERMS

A signup form is only supposed to let you set a few things: your name, phone, email, organization. Behind the scenes, the server writes a new row into a database table that also has other columns you were never shown — things like which plan you're on, whether your phone is verified, how many days your trial has left, or your account's role. A well-built form only ever writes the fields it showed you. A poorly-built one sometimes blindly writes whatever fields it's given — so if you can sneak extra fields into the request, you might be able to set values you were never supposed to touch.

1 **YOU — DEVTOOLS**

Go through signup normally, find the actual sign-up submission request in the Network tab, right-click → **Edit and Resend**.

2 **YOU — DEVTOOLS**

Look at the request body. Remember P2Less's forms are Next.js Server Actions, so the body is a plain ordered list of argument values rather than named JSON fields — you're looking for a place to *add* to that list, not to find a field by name.

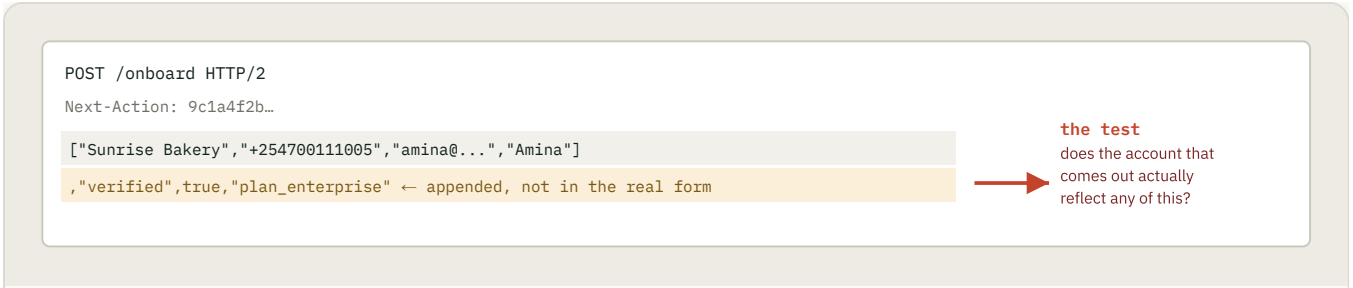
3 **YOU — BURP SUITE**

This is easier with more room to work: send the same request to Repeater instead. Try appending extra values onto the end of the argument list — for instance, an extra `true` (testing for something like "already verified"), an extra number far above any real plan's limit (testing for something like a pre-set balance or message allowance), or a recognizable plan name/id copied from a plan you haven't paid for (testing for something like being assigned straight onto a paid plan). Send it.

> **Expect:** the extra values are silently ignored and have zero effect on the resulting account — it should come out exactly as if you'd sent the normal, unmodified request. If your account instead comes out already verified, on a paid plan, or with an inflated balance you never bought, that's a serious, reportable mass-assignment bug.

4 **YOU**

Whatever the outcome, check the resulting account for every field you tried to influence — the trial dates, the plan, the verification status, the message/AI balances — not just the one thing that prompted the test.



Appending extra values onto the sign-up request's real argument list, in Repeater — the mass-assignment test in practice.

VECTOR 10

Mass account creation with a browser extension

A real abuser doesn't sign up once by hand — they install a free browser extension, record the sign-up flow one time, and let it replay itself dozens of times unattended. This is that exact tool, used the exact way.

1 YOU — SELENIUM IDE

Go to the Chrome Web Store, search **Selenium IDE** (official, published by SeleniumHQ), click **Add to Chrome**.

2 YOU — SELENIUM IDE

Click its new toolbar icon. Choose **Create a new project**, name it `P2Less mass signup test`.

3 YOU — SELENIUM IDE

Click the red **Record** button. A new browser window opens, already tracked by the extension.

4 YOU — SELENIUM IDE

In that window, go through one entire P2Less signup by hand — organization name, phone, name, email, submit, enter the on-screen demo OTP, verify.

> **Expect:** every click and every typed character appears as a new row in Selenium IDE's command list as you go.

5 YOU — SELENIUM IDE

Click **Stop recording** when you land on the dashboard.

6 YOU — SELENIUM IDE

In the recorded command list, find the row that typed your organization name, and the row that typed your phone number. Change each one's value to a placeholder like `${orgName}` and `${phone}`.

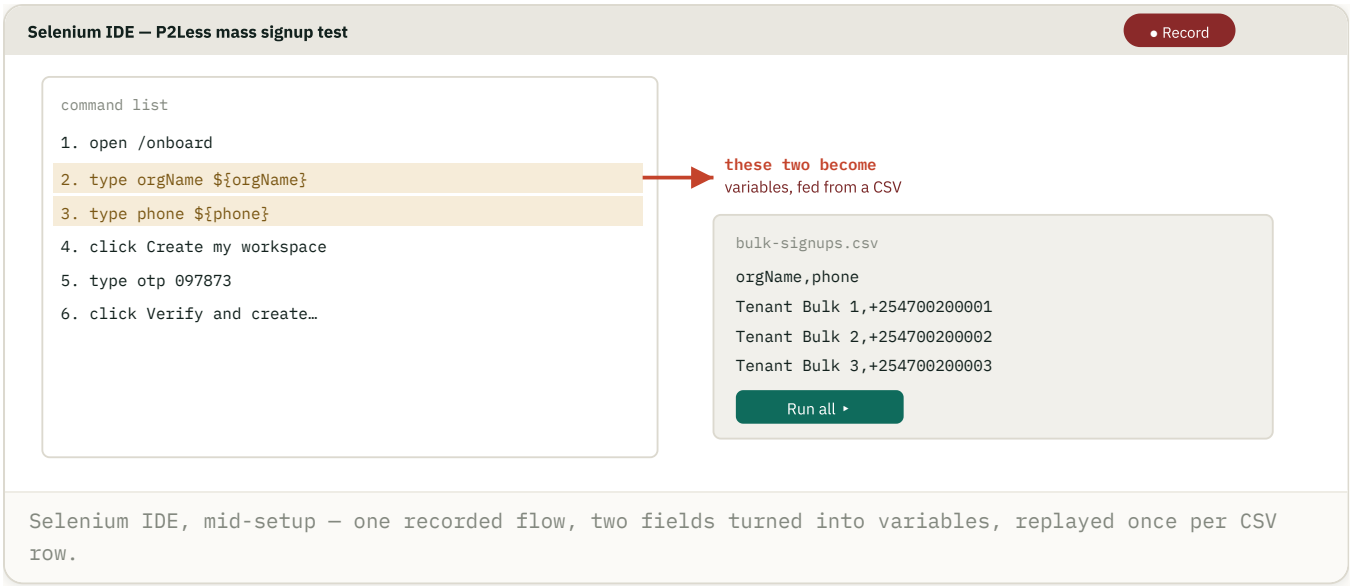
7 YOU — SELENIUM IDE

Click the project's menu → **Data-driven testing** (or the equivalent "Run with data" option in your version) and load a small CSV file with a few rows, each providing a different `orgName` and `phone` value — e.g. `Tenant Bulk 1,+254700200001`, `Tenant Bulk 2,+254700200002`, and a handful more.

8 YOU — SELENIUM IDE

Click **Run** (or **Run all**) to replay the recorded signup once per row in your data file, unattended.

> **Expect & what to check afterward:** the real question isn't whether the macro runs — it's what P2Less does about it. Check: did every single one succeed and get a full 7-day trial with no friction at all (a real gap — this exact tool is free and takes ten minutes to set up)? Did the system slow down, block, or flag the pattern after some number of rapid, near-identical signups from the same browser/IP? Is there any rate limit at all on the signup endpoint itself, separate from the OTP one tested in Vector 9?



Selenium IDE, mid-setup — one recorded flow, two fields turned into variables, replayed once per CSV row.

WHY THIS ONE MATTERS AS MUCH AS IT DOES

Every technique elsewhere in this manual takes a person, sitting there, clicking. This is the one that takes a person ten minutes to set up once and then walks away. If nothing catches a dozen near-identical signups fired back to back with no delay, an actual abuser could do this at a scale far beyond anything one afternoon of manual testing could ever show you.

REAL REQUEST TAMPERING

Now open the toolbox for real

The highest-value section for money, plans, and identity specifically. Each technique is shown both the plain-DevTools way and the Burp way.

A REAL DETAIL ABOUT HOW P2LESS'S FORMS WORK

P2Less is built on Next.js, and most of its buttons (upgrade, top-up, connect, save) use a **Server Action** — the browser sends a plain `POST` to the SAME page address you're already on, carrying a `Next-Action` header, with the actual arguments packed into the body as a plain, readable, ordered piece of text rather than named JSON fields like `{"planId": "..."}`. Don't look for a field literally called `planId`. Look for anything in that body text you recognize — a number you just typed, the exact plan name you clicked, an amount matching what was on screen — and edit that value in place.

1. Tamper with dates

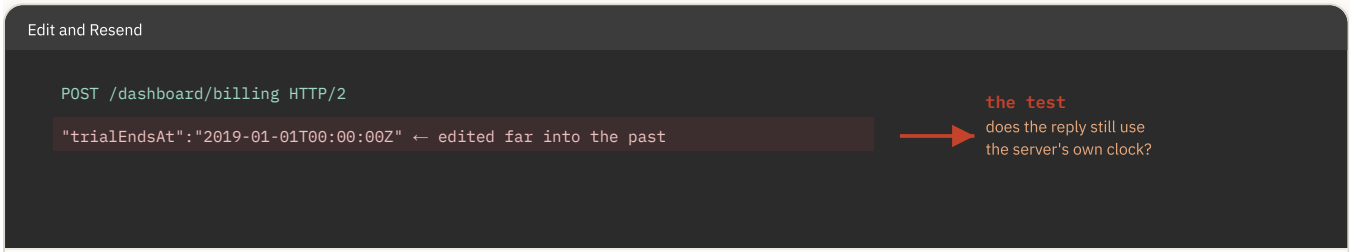
1 **YOU — DEVTOOLS**

Open the upgrade modal on Tenant 5. Find the request that fires when it opens (a `POST` to the current billing URL with a `Next-Action` header). Right-click → **Edit and Resend**.

2 **YOU — DEVTOOLS**

Find anything resembling a date/timestamp in the body, change it far into the past or future, keep the surrounding punctuation identical, click **Send**.

> **Expect:** the response still reflects the server's own real clock, never the date you supplied.



A date field edited client-side before sending — billing logic should never trust this.

2. Tamper with plan or price mid-flight

1 **YOU — BURP SUITE**

Start an upgrade to the most expensive plan. Find that request in **Proxy → HTTP history**, send to Repeater.

2 **YOU — BURP SUITE**

Swap the plan's identifying text for a cheaper plan's, or lower any number matching a price you saw on screen. Send.

> **Expect:** the server recomputes the true price itself from its own database using only the plan id it trusts — the invoice should come back unchanged and correct, or be rejected outright.

3. Tamper with the payment amount

1 **YOU — DEVTOOLS**

Trigger a payment submission. Find it, **Edit and Resend**, lower the amount field below what's actually owed.

> **Expect:** rejected outright, or the invoice correctly stays "partially paid" — the plan must never upgrade on a client-invented amount.

4. Tamper with identity and session

1 **YOU — DEVTOOLS**

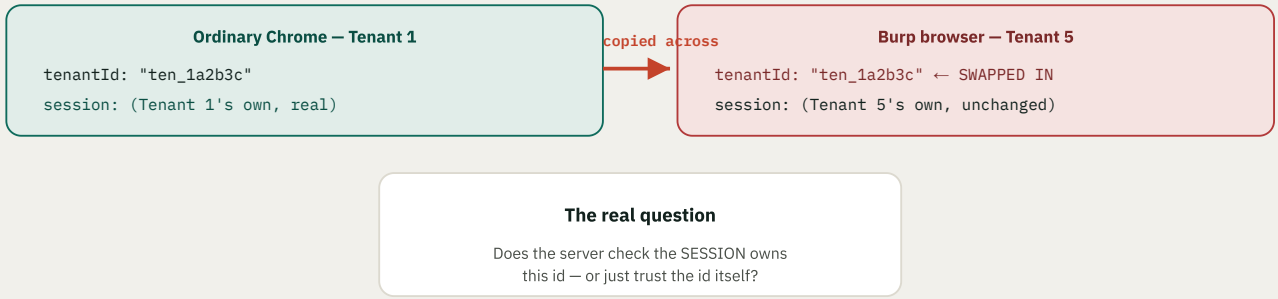
While logged into Tenant 5, edit a single character of the session cookie (**Application → Cookies**), reload.

> **Expect:** logged out or refused.

2 **YOU — BURP SUITE**

Log in as Tenant 1 in ordinary Chrome, Tenant 5 in the Burp browser. Find a Tenant 5 request whose body contains a tenant/subscription id, send to Repeater, substitute Tenant 1's id while keeping Tenant 5's session cookie. Send.

> **Expect:** refused. The server must check the logged-in session actually owns the id inside the body, not just trust whatever id the client sends — the same class of gap as a real, historical bug already found and fixed once in this codebase. If this succeeds, it's the single highest-severity finding this manual could produce.



The highest-severity test in this whole manual — Tenant 5's own valid session, carrying Tenant 1's id.

5. Tamper generally — observe everything

1 **YOU — BURP SUITE**

Spend 20 minutes clicking through Tenant 5's dashboard with Burp recording. Send five or six varied requests to Repeater, try small edits to any numeric field, id, or boolean you find. No single expected outcome — the goal is coverage.

IF ANY OF THE FIVE ACTUALLY WORK

Stop. Save the exact request (Burp: **Copy as curl command**; DevTools: **Copy → Copy as cURL**) and the exact unexpected response. Treat it as a real, urgent bug. Don't keep probing the same hole — report it.

THE HARDEST TEST

Suspended — trying to get back in

P2Less's admins can suspend a tenant they've spotted abusing the system. This chapter tries every realistic way a suspended user might claw their way back.

1 **TECH — ADMIN CONSOLE**

A teammate, logged in as admin, suspends Tenant 5 from `/admin/tenants`.

> **Expect:** the status visibly changes in the admin list, with a real, audited log entry recording who and when.

2 **YOU**

In a tab where you were already logged in before the suspension, click any dashboard link without reloading first.

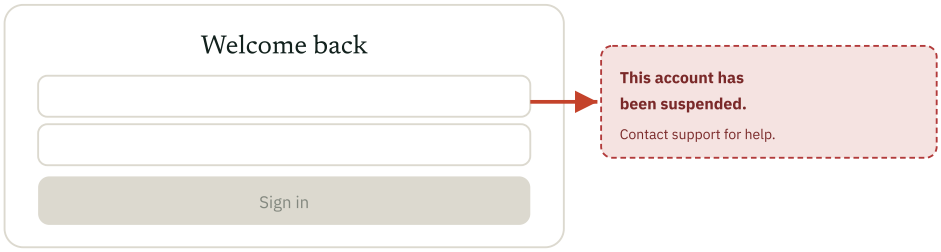
> **Expect:** blocked immediately, with a clear message — testing whether suspension cuts you off mid-session, not just at the next login.

3 **YOU**

Try a full fresh login.

> **Expect:** a specific, honest "account suspended" message, not a generic wrong-password error.

.../login



What a correct, honest suspension message looks like — specific, not a generic "wrong password."

4 **YOU — DEVTOOLS**

If you saved Tenant 5's old session cookie value beforehand, add it manually to a fresh Incognito window's Cookies for P2Less's address.

> **Expect:** refused too — suspension should invalidate the session itself, not just block the login form.

5 **YOU**

Try **Forgot password** with Tenant 5's real email, all the way through.

> **Expect:** either the reset is blocked, or it succeeds but landing in the dashboard is still blocked the same way — a reset must never be a side door.

6 **YOU — BURP SUITE**

Replay an old captured Tenant 5 request from Repeater's history.

> **Expect:** refused with the same suspended-account error everywhere — the real test of whether suspension is checked on every action, not just at login.

7 **YOU**

Sign up fresh, reusing Tenant 5's phone but a new email (or vice versa).

> **Expect:** the suspension follows the identity, not just the account record.

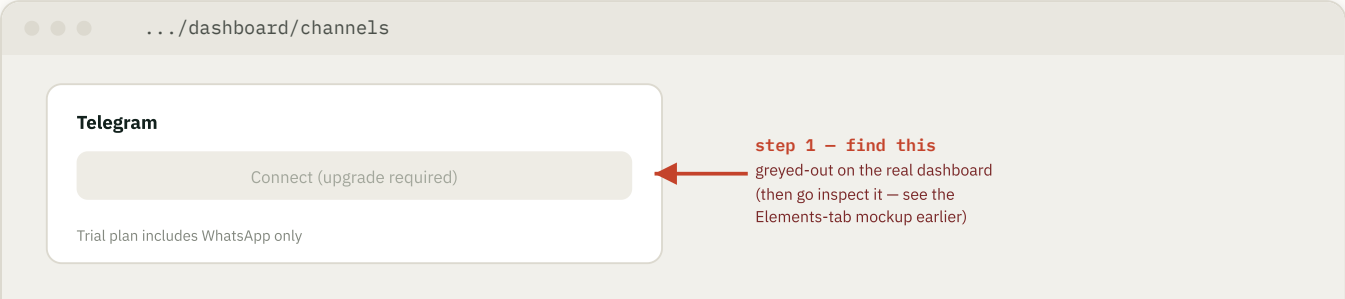
8 **YOU**

If any "Contact support" or appeal link exists on the blocked screens, use it and note exactly what happens.

THE OTHER HARDEST TEST

Re-enabling a disabled button or channel

Sometimes P2Less turns a control off in the UI. This chapter tries to turn it back on from outside the rules.



A real greyed-out control, on the actual dashboard — this is what you're hunting for before opening DevTools.

1 **YOU**
Find a greyed-out control on Tenant 5's dashboard — a channel above this plan's allowance, or an action blocked by a usage limit.

2 **YOU — DEVTOOLS**
Element picker (`Ctrl+Shift+C`), click the control.

> **Expect:** Elements jumps to its exact tag.

3 **YOU — DEVTOOLS**
Look for a bare `disabled` attribute, `aria-disabled="true"` , or a class like `opacity-50` / `pointer-events-none` / `cursor-not-allowed` .

4 **YOU — DEVTOOLS**
If it's a bare `disabled` attribute: double-click it, delete it, press `Enter` .

> **Expect:** visually clickable again immediately.

5 **YOU — DEVTOOLS**
If it's class-based: in the **Styles** panel, find the rule setting opacity/pointer-events, strike it out.

6 **YOU — DEVTOOLS**
Click the control, watch the Network tab.

> **Two outcomes, both meaningful:** (a) a real request fires, and either succeeds (a genuine bug) or is rejected server-side (correct — the UI disable was only a hint); or (b) nothing happens — no new row at all.

7 **YOU — DEVTOOLS / BURP SUITE**

If (b): find a request from a moment an *allowed* tenant performed the equivalent action (e.g. Tenant 1 connecting a plan-included channel). **Edit and Resend** or send to Repeater, keep Tenant 5's session cookie, swap in the restricted feature. Send.

> **Expect:** refused by the server. If it succeeds, the restriction only ever existed as a greyed-out button — a serious, reportable gap, since anyone with DevTools open (free, built into every browser) could find this the same way you just did.

VECTOR 11 — FORWARD-LOOKING

Face verification, tested before it ships

Not built yet — keep this chapter ready

P2Less doesn't have face verification today. This chapter exists so the day it does, this manual doesn't need to be rewritten — just re-run against the real feature the same way everything else above already was.

WHY THIS MATTERS AS MUCH AS OTP DOES

Face verification exists to prove a real, specific, live person is present. Every known real-world attack against it targets exactly one of those three words: is it really a face (not a photo), is it really that person (not someone else's), and is it really happening now (not a recording).

When the day comes, test these in order

The four questions this whole chapter reduces to — a conceptual illustration for a feature that doesn't exist yet.

1 **YOU**

The photo test. Print a clear, well-lit photo of a face (yours or a colleague's, with consent), hold it up to the camera during verification instead of your live face.

> **Expect:** rejected — a system with real liveness detection should notice the complete absence of natural micro-movement, blinking, and depth.

2 **YOU**

The screen-replay test. Play a recorded video of a face on a second phone or tablet, held up to the camera in place of a live feed.

> **Expect:** rejected — screen glare, refresh-rate flicker, and the video's fixed loop should all be things real liveness detection is built to catch.

3 **YOU**

The wrong-person test. With one person's account already verified, have a genuinely different person attempt to pass that same account's ongoing/repeat verification.

> **Expect:** rejected — whatever reference image or embedding was captured at enrollment must actually be compared against, not just "a face was present."

4 YOU

The liveness-prompt-replay test. If verification asks for an action (blink, turn your head, smile), record that exact sequence once, then try replaying the recording for a second, separate verification attempt.

> **Expect:** rejected, or the system asks for a different, unpredictable sequence each time specifically so an old recording can't be replayed.

5 YOU — DEVTOOLS/BURP SUITE

The in-flight swap. Let the camera capture happen normally, but intercept the actual upload request afterward (Edit-and-Resend, or Burp Repeater) and swap the submitted image file for a different one before it reaches the server.

> **Expect:** rejected, or ideally structurally impossible — the strongest designs never let a raw image file travel client-side at all between capture and server-side check.

6 YOU

The storage question. Once verification is live, ask directly (of the system, and of whoever built it): where is the captured face data stored, is it encrypted, who can access it, and can a tenant request it be deleted? Biometric data deserves a straight answer to all four before this feature ships to real customers, not just after.

VECTOR 12

Inflating your own message or AI balance

Distinct from paying too little (Vector 4 and Real request tampering Technique 3): this chapter tests whether the balance number itself — the thing that actually gates whether a message can send — can be pushed up without a real payment behind it at all, or credited more than once for a single real payment.

WHY THIS IS ITS OWN VECTOR, NOT JUST "MONEY TAMPERING" AGAIN

P2Less's real design keeps the balance as the single source of truth for whether sends are allowed, and only ever increases it inside the one function that also verifies a matching real payment exists. This chapter exists to test that specific design decision directly — not the price shown on an invoice, but the number that actually gets written to the account.

STEP A — PUSH THE TOP-UP NUMBER PAST WHAT THE UI ALLOWS

1 **YOU**

Open the upgrade modal, move the extra-messages slider to its maximum, and note the highest number the slider itself will let you pick.

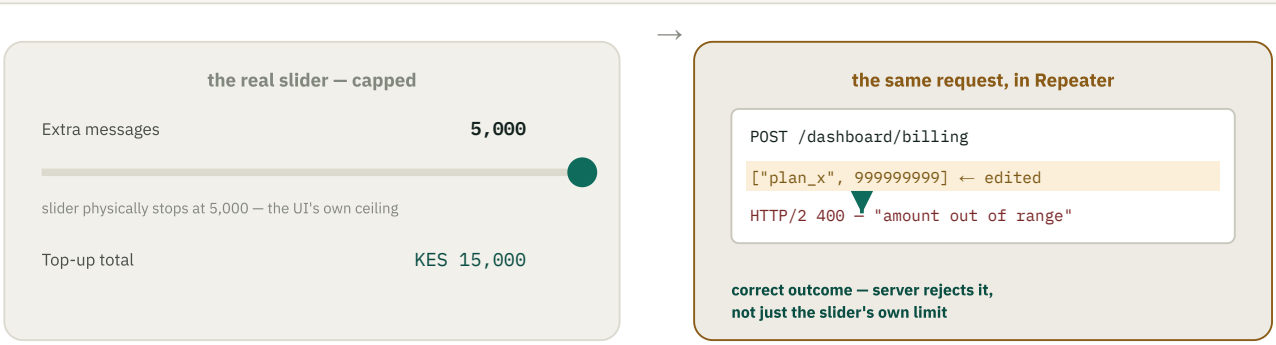
2 **YOU — BURP SUITE**

Start the top-up at that maximum, find the resulting invoice-creation request in Burp's history, send it to Repeater.

3 **YOU — BURP SUITE**

In the body, find the number matching the slider value you picked and replace it with something far larger — e.g. `999999999`. Send.

> **Expect:** the server clamps or rejects the out-of-range number rather than trusting it — the resulting invoice amount and any credited balance should never reflect a number the real UI would never have allowed you to pick.



The slider's own limit vs. what happens when that limit is bypassed directly in the request — the server has to be the real gate, not the slider.

STEP B — TRY TO CREDIT THE BALANCE WITH NO REAL PAYMENT AT ALL

1 **YOU — BURP SUITE**
Start a top-up or upgrade so the invoice exists, but stop right before submitting any payment method.

2 **YOU — BURP SUITE**
In Burp's history, look for whatever request fires once a payment is confirmed (in demo mode, this is the request right after clicking **Pay** that produces "Recorded (demo mode)… Your upgrade is now active"). Send that specific request to Repeater.

3 **YOU — BURP SUITE**
Send it again, now, directly — without ever having gone through the actual payment step for this particular invoice.

> **Expect:** rejected, or it has no effect, because the real underlying check is a genuine sum of confirmed `Payment` records against what's actually owed — a status flip alone, without a matching payment record behind it, should never be enough to move the balance.

STEP C — TRY TO GET CREDITED TWICE FOR ONE REAL PAYMENT

1 **YOU**
Complete one real (or demo-mode) top-up all the way through to "Payment confirmed," and note the resulting balance shown on the dashboard.

2 **YOU — BURP SUITE**
Find that same confirmation request in Burp's history — the one that just succeeded — and send it to Repeater. Click **Send** two or three more times in a row.

3 **YOU**
Reload the real dashboard and check the balance again.

> **Expect:** unchanged from Step C.1's number — a real compare-and-swap on the invoice's own status should mean the second and third replay find the invoice already settled and do nothing further, no matter how many times the confirmation is resent.

STEP D — TWO TABS AT ONCE (A RACE CONDITION)

1 **YOU**
Open the same pending invoice's payment step in two separate browser tabs, side by side.

2 **YOU**
Trigger the confirmation in both tabs as close together in time as you can manage — two clicks, back to back, as fast as possible.

> **Expect:** only one credit is ever applied, not two — if the two nearly-simultaneous requests both slipped past a status check before either had written its result, that's a real race-condition bug worth reporting, distinct from the simpler sequential-replay test in Step C.

STEP E — A QUICK SANITY CHECK: THE DISPLAY ISN'T THE TRUTH

1 **YOU — DEVTOOLS**

On the dashboard, use the element picker to find the on-screen text showing your current balance, and directly edit its text via **Edit as HTML** to a large number.

> **Expect:** this changes nothing real — it's purely cosmetic in your own browser. Reload the page and it reverts to the true, server-held number. This step exists only to confirm, plainly, that the display is not where the real balance lives — the meaningful tests are B, C, and D above, not this one.

WHAT "GOOD" LOOKS LIKE ACROSS THIS WHOLE VECTOR

- The number the server actually enforces sends against is never influenced by anything the client merely displays or requests — only by a genuinely verified payment.
- One real payment can never be replayed into two credits.
- Two near-simultaneous confirmations for the same invoice never both succeed.

RESULT LOG

Record every attempt here

VECTOR	WHAT YOU TRIED	TOOL USED	EXPECTED	ACTUAL	HELD / BROKE
Not started — nothing logged yet.					